

Design and Development of a Small-Scale All-Terrain Ground Vehicle for Autonomous Operation

Miguel Pereira Francisco Pinho
miguel.p.pinho@tecnico.ulisboa.pt

Instituto Superior Técnico, Universidade de Lisboa, Lisboa, Portugal

July 2025

Abstract

Mobile robots are an emerging technology with expanding applications across various fields, including defense. In response to this trend, the Portuguese Army has invested in initiatives like the EXE03 project. This thesis contributes to that effort by supporting the development of an Uncrewed Ground Vehicle (UGV). This vehicle, designed at scale, constitutes an experimental platform for the development and validation of solutions aimed at autonomous vehicles in real environments.

The objectives of the project were defined based on functional requirements outlined by the European Defence Fund for military UGVs, focusing on teleoperation, waypoint navigation, and collision detection and avoidance. A functional differential drive robot was designed and built from the ground up, based on the Devastator platform and equipped with a sensor suite aligned with the defined objectives.

A ROS2 based software framework was implemented and containerized with Docker to ensure reproducibility and scalability across different platforms. A collision avoidance algorithm was developed to assist teleoperation by applying corrective velocity commands, based on obstacle proximity.

The UGV is SLAM capable through the integration of the SLAM Toolbox package. This functionality was validated in both indoor and outdoor environments, exhibiting accurate localization and mapping performance.

A functional waypoint navigation solution was implemented, utilizing the Navigation 2 framework. Three different local planners were tested, in a challenging scenario, where waypoints were assigned randomly. A total of 60 waypoints were reached, demonstrating reliable autonomous navigation.

Keywords: Uncrewed Ground Vehicle, Autonomous Navigation, Collision Avoidance, Simultaneous Localisation and Mapping, Robotics, All-terrain

1. Introduction

In an era of rapid technological advancements, autonomous mobile robots are becoming increasingly integrated into various aspects of our day to day lives. Their ability to navigate dynamic environments, make real-time decisions, and operate with minimal human intervention has made them indispensable tools in modern applications. With continued developments in artificial intelligence, sensor technology, computer power and communications, mobile robots are poised to play a central role in shaping the future of automation and intelligent systems, including in defence, where Uncrewed Ground Vehicles (UGVs) are increasingly used in a variety of combat and support missions.

A UGV can be described as a comprehensive intelligent system that integrates environmental perception, localization, navigation, path planning, decision-making and motion control [1]. UGVs offer capabilities that support a wide range of military operations, including combat, intelligence, surveil-

lance and reconnaissance (ISR), logistics, and explosives or mine disposal. These strategic applications have led organizations like the European Union and NATO to invest heavily in UGV development, recognizing them as one of the key technologies by 2035 [2].

Portugal, through the Portuguese Army in partnership with Instituto Superior Técnico, launched the EXE03 - Uncrewed Ground Systems Project. One of the work packages within this project is *UGVSecurity*. This master's thesis aligns with this work package, which aims to develop a transportable UGV for active security of critical infrastructures and detection of objects of interest, while meeting the functional requirements for military UGVs outlined in the European Defence Fund's Commission Implementing Decision [3].

The project objectives were defined by distilling these requirements into three core capabilities: teleoperation, enabling remote human control; waypoint navigation, allowing autonomous travel be-

tween predefined points; and collision avoidance, a feature integrated across all modes to ensure safety throughout all UGV operations.

2. The Devastator Platform

To meet these objectives, a compact and modular UGV was developed based on the tracked differential drive Devastator platform.

2.1. Robot Platform and Hardware Components

The Devastator platform was chosen for its compact size (225*220*108 mm), lightweight (780 g) aluminum alloy frame, and high-performance suspension, ensuring mobility across challenging terrains while supporting various components. Two FIT0493 DC motors, each with a 34:1 gearbox and quadrature encoder, drive the UGV, controlled via PWM signals through a motor driver. The Nvidia Jetson TX2, paired with the Auvidia J121 carrier board, serves as the onboard computer, providing a 6-core ARM CPU, 256-core Pascal GPU and 8 GB LPDDR4 RAM memory, for computationally intensive tasks like SLAM and path planning.

For environmental perception, the Hokuyo URG-04LX-UG01 LiDAR sensor offers 240° 2D scanning with a 4 m range. Two LiPo batteries power the system: one for the motors and driver, and another for the Jetson and peripherals. An Arduino Duemilanove interfaces with the motor driver, compensating for the J121's lack of GPIO pins, while a powered USB hub connects and supplies the necessary current to the peripherals.

2.2. Robot Assembly

Fig. 1 illustrates the robot's components circuit.

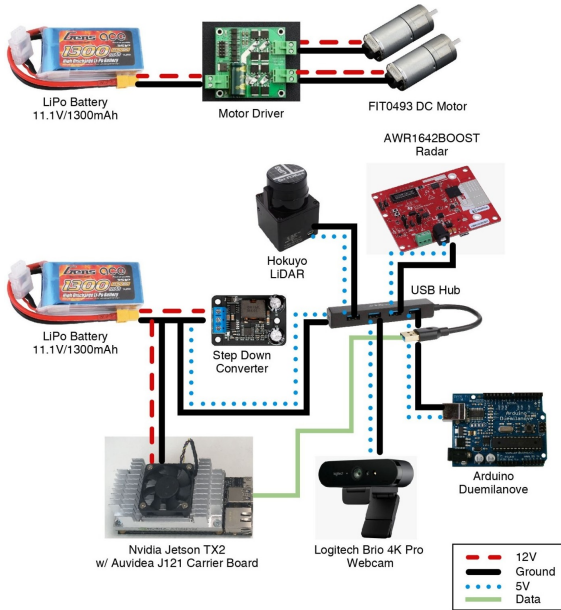


Figure 1: UGV's components electrical circuit.

Component placement was optimized using a

3D CAD model in SolidWorks, ensuring compliance with spatial constraints. Custom 3D-printed mounts were designed to interface the Devastator's mounting holes with the external components. Fig. 2 showcases the final assembly of the UGV, from multiple views.

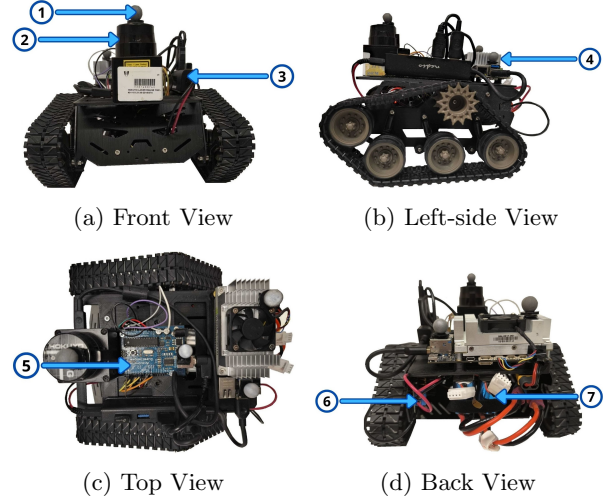


Figure 2: The Devastator robot. Components labelled in Table 1.

Table 1: UGV's main components.

Identifier	Component Name
1	Reflective Marker
2	Hokuyo URG-04LX-UG01 LiDAR
3	USB Hub
4	Jetson TX2 w/ J121 Board
5	Arduino Duemilanove
6	Step Down Converter
7	LiPo Batteries

2.3. System Software and Virtualization Tools

ROS2 [4] was used as the state of the art open source robotics middleware. The computing hardware of the UGV's setup includes a host computer, in addition to the Nvidia Jetson TX2, which provides the interface through which the operator commands and visualises the robot's tasks. The Linux distribution utilized in the host computer was Ubuntu 22.04.5., on a virtual machine. On the Jetson TX2, Ubuntu 20.04.6 is the most recent distribution compatible, creating a compatibility issue, as the newest ROS2 version at the time of development (Humble Hawksbill) is only supported by 22.04 or newer versions.

This problem was solved through containerization using Docker, combining the application's code, dependencies, and system libraries in the form of containers. Docker increases code repro-

ducibility, enabling the developed features to be easily deployed across different robotic platforms. A Dockerfile was crafted to build a Docker image for the UGV's ROS2 Humble workspace, based on the `arm64v8/ros:humble-ros-core` image for ARM64 architecture.

Sensor data was visualized in RViz and Foxglove Studio via ROS2 topics. A digital model of the UGV was created using the `solidworks_urdf_exporter` [5], which converted the UGV's CAD assembly into a URDF file for realistic virtual representation. Although this ensured high visual fidelity, the complex mesh files caused performance issues during real-time visualization due to hardware limitations. To address this, a simplified model using basic geometric shapes was adopted, retaining the correct dimensions and sensor layout while improving performance.

Gazebo was chosen for simulation due to its open-source nature and strong ROS2 integration, enabling realistic physics-based testing without risking hardware. A simplified robot model was used in custom lightweight Gazebo worlds for efficient core functionality testing.

3. Implementation

3.1. Low-Level Control

In the context of this work, low-level control entails receiving target motor speeds and actuating real motors accordingly. Fig. 3 shows the components of the control system and how they interact.

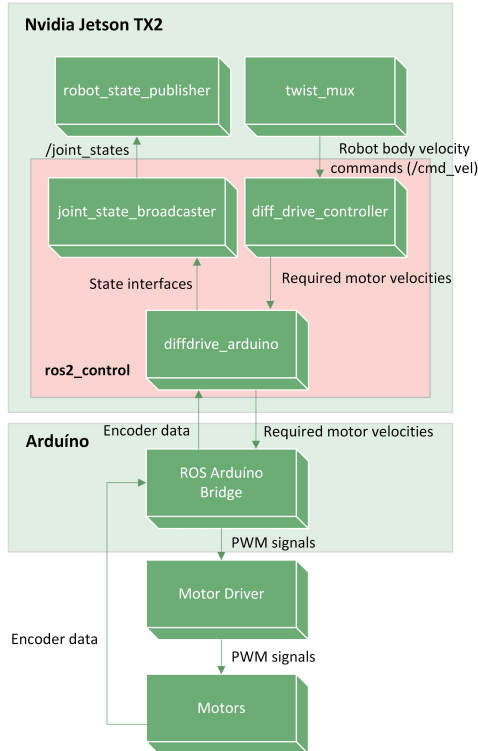


Figure 3: Control system architecture.

3.1.1 Arduino Motor Controller

Since the Auvideo board lacks pins capable of delivering PWM signals, an Arduino acts as an intermediate board, receiving velocity commands via USB serial and sending PWM signals to the motor driver. It executes `ros_arduino_bridge` [6] with closed-loop PID control based on encoder feedback.

3.1.2 Differential Drive Robot Kinematics

In ROS2, it is standard to define velocities using the `geometry_msgs/Twist` message type. This message type defines the robot's velocity as a combination of its body's linear velocity in x , and its angular velocity around z (yaw). This message must be translated into individual wheel velocities, which can be obtained from the kinematic model of the robot. The kinematic model of a wheeled drive robot relies on the rolling and sliding constraints. Siegwart et al. [7] demonstrate how the equations obtained by these constraints allow for the development of a kinematic model. For a wheeled differential drive vehicle, the model obtained defines how the robot's linear ($\dot{x}$) and angular velocities (ω) can be calculated as the result of the left (ω_L) and right wheel angular velocities (ω_R):

$$\dot{x} = \frac{r(\omega_R + \omega_L)}{2}, \quad \omega = \frac{r(\omega_R - \omega_L)}{b} \quad (1)$$

where r is the wheel radius and b is the distance between the right and left wheels. In order to obtain the wheel angular velocities necessary to produce a desired linear and angular velocity for the robot, the inverse kinematics are utilized:

$$\omega_R = \frac{\dot{x} + \frac{b}{2}\omega}{r}, \quad \omega_L = \frac{\dot{x} - \frac{b}{2}\omega}{r} \quad (2)$$

For tracked vehicles, standard wheel constraints do not apply. However, the wheeled vehicle kinematic model can still be used as an approximation.

3.1.3 ROS2 Control Framework

The ROS2 Control framework [8] was utilized, enabling real-time control of the UGV by bridging software and hardware. It processes velocity commands and converts them into wheel angular velocities using the robot kinematics, while leveraging wheel encoder feedback to publish odometry. The framework's architecture includes:

- **Controllers:** The `diff_drive_controller` computes wheel commands from robot body velocity inputs, based on the kinematic equations;
- **Hardware Interface:** The `diffdrive_arduino` interface [9] connects the controller to the Arduino, configured via the robot's URDF file to specify serial communication and joint states.

- **Controller and Resource Managers:** These manage multiple controllers and hardware interfaces, ensuring seamless lifecycle operations and resource allocation.
- **Broadcasters:** The `joint_state_broadcaster` publishes joint states to `/joint_states`, enabling visualization in tools like Rviz.

3.2. Teleoperation

The robot's teleoperation was achieved using a Dualshock 4 gamepad, selected for its ergonomic design and dual analog sticks. The left stick controls linear velocity, the right stick controls angular velocity, and the L1 button activates commands to prevent interference with lower-priority sources. The gamepad connects to the host computer, which communicates velocity commands to the Jetson TX2 via Wi-Fi, extending operational range beyond Bluetooth limits. In ROS2, the `joy` node detects gamepad inputs, while the `teleop_twist_joy` node converts the thumbstick data into `Twist` messages published as `/cmd_vel_joy`.

3.3. Collision Avoidance Algorithm

ROS2 based implementations of reactive collision avoidance techniques include Nav2's collision monitor [10], which slows down or stops the UGV after a set number of obstacle points are detected by LiDAR scans within a designated zone. Another example is the TurtleBot3 Obstacle Avoidance application [11], which issues a rotational command to turn the robot away if an obstacle exists within a defined angular field in front of the robot.

An algorithm was developed that integrates key elements of both systems referenced. It defines multiple safety zones around the robot (Fig. 4a), each triggering increasingly conservative responses as obstacles are detected closer to the robot. The algorithm incorporates rotational commands, allowing the robot to reorient itself away from hazards. Fig. 4b illustrates the sequence of actions employed.

Three zones ahead of the LiDAR are defined (1 to 3), representing decreasing levels of urgency. For each of the zones, the algorithm iterates through the LiDAR `/scan` messages to find the closest reading within that zone, identifying the nearest obstacle and its angle. If an obstacle is detected and a forward velocity command is given, a correction to that command is applied. This correction is applied to the linear velocity $\hat{x}$ and the angular velocity ω , resulting in a new command $(\hat{x}, \hat{\omega})$, published on the `/cmd_vel_stop` topic. Given:

- d : the distance to the closest obstacle in a zone;
- $\phi \in (-\phi_{\max_j}, \phi_{\max_j})$: the angle of the closest obstacle, relative to the forward axis;
- α_j and k_j : user defined constants.

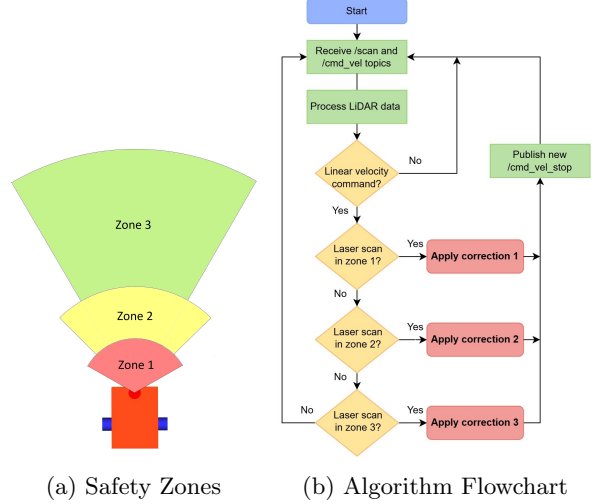


Figure 4: Collision Avoidance Algorithm.

The correction applied to the linear and angular velocities for Zone 1 is:

$$\hat{x} = 0, \quad \hat{\omega} = \alpha_1 \cdot \omega \quad (3)$$

In Zone 1, the linear velocity is immediately halted to prevent a collision, with only negative linear velocity commands being permitted. The angular velocity is reduced proportionally to the user's command, resorting to a scaling constant α_1 . This allows the user to maintain control of the UGV and slowly steer it away from danger. The correction applied in Zone 2 is:

$$\hat{x} = k_2 \cdot \dot{x} \cdot d \quad (4)$$

$$\hat{\omega} = \begin{cases} \min(\alpha_2(-\pi/2 + \phi), \omega), & \text{if } \phi \geq 0 \\ \max(\alpha_2(\pi/2 + \phi), \omega), & \text{if } \phi < 0 \end{cases} \quad (5)$$

Here, the linear velocity command is scaled down. Dependence on obstacle distance d allows velocity to decrease gradually as the robot nears the obstacle, smoothing the transition between Zones 2 and 3. The angular velocity correction steers the UGV away from the obstacle, faster when the obstacle is head on ($\phi = 0$). The $\min$ and $\max$ conditions prioritize the user given rotation command, if larger and in the same direction of the corrective command. For Zone 3, the analogous corrections are:

$$\hat{x} = k_3 \cdot \dot{x} \cdot d \quad (6)$$

$$\hat{\omega} = \begin{cases} \min(\alpha_3(-\pi/4 + \phi), \omega), & \text{if } \phi \geq 0 \\ \max(\alpha_3(\pi/4 + \phi), \omega), & \text{if } \phi < 0 \end{cases} \quad (7)$$

The k_1 and k_2 constants can be tuned to apply a stricter linear velocity correction for Zone 2.

To manage multiple simultaneous velocity command sources, a priority scheme was implemented

using `twist_mux` [12]. Collision avoidance commands were given highest priority as a safety measure, followed by gamepad inputs and finally the Nav2 stack commands. Placing the gamepad above the commands provided by the Nav2 stack ensures manual override by the user is always possible.

3.4. SLAM

To enable navigation in GPS-denied environments, the robot employs SLAM Toolbox [13], a ROS2 graph-based SLAM package for simultaneous localization and mapping. A thorough review of SLAM packages was performed, and priority was given to the use of 2D LiDAR-based SLAM solutions over Visual SLAM, due to the higher map accuracy and overall robustness. SLAM Toolbox was favoured over other SLAM packages, given its superior performance, widespread use in production grade robotics projects and active support. SLAM Toolbox was used in online asynchronous mode, prioritizing the latest LiDAR scan to reduce delays, crucial for real-time operation.

3.5. Waypoint Navigation

The robot’s autonomous waypoint navigation leverages the Navigation 2 (Nav2) framework [14], which provides a platform for integration of path planners and controllers, with additional features. Nav2 uses behavior trees to manage navigation tasks, orchestrating multiple modular servers that handle specific functions like path planning, control, and recovery, which communicate via ROS action servers.

Nav2 models the environment using two grid-based costmaps (local and global) composed of modular layers. The global costmap’s base layer, `StaticLayer`, uses the SLAM map to define the environment. The `InflationLayer` exponentially inflates the lethal costs of the occupied space, turning a tri-nary occupation grid into a cost grid ranging from 0 to 255. The layer accomplishes this by expanding outwards from obstacles to a given distance, using:

$$cost(x, y) = (cost_{lethal} - 1)e^{-\omega_{scale}(d_o - r_f)} \quad (8)$$

where ω_{scale} is the cost scaling factor, d_o is the distance to the obstacle, $cost_{lethal}$ is the lethal obstacle cost, and r_f is the inscribed radius of the footprint. The local costmap’s base layer is the `ObstacleLayer`, which creates the local occupancy grid using live LiDAR scans.

Nav2 provides several global path planner plugins. The Navigation Function planner (NavFn) uses Dijkstra’s Algorithm to expand a potential field, calculated using a quadratic kernel function, from the goal cell to the start. The kernel at a cell is determined by its cost and the neighboring cells’ potential. Costs are linearly adjusted from the cost grid to create a neutral-to-maximum range, generating increasing wavefront costs until the start cell

is located. The path is traced back using gradient descent, halving resolution increments for a smooth and uniform path. Nav2 maintainers [15] recommend this planner for differential drive robots able to rotate in place anywhere in its environment, making it suitable for this implementation.

Nav2 also provides a selection of controller plugins. The **DWB** controller implements the Dynamic Window Approach, which samples velocity commands from the dynamic window of possible velocities, obtained through the robot’s dynamic limits. These velocities generate a set of circular arcs as candidate trajectories, which are then evaluated based on critical functions, selecting the best.

The **Regulated Pure Pursuit (RPP)** controller aims to simply follow the path computed by the global planner without deviation. Pure Pursuit tracks a reference path by computing a lookahead point at a fixed distance from the robot, which determines the curvature and steering commands needed to follow the path. RPP enhances this by incorporating dynamic adjustments to the lookahead distance based on the robot’s velocity, ensuring smoother tracking in varying conditions.

Vector Pursuit is a geometric path-tracking method using screw theory. It calculates two instantaneous screws: one for translation and one for rotation. These screws combine into a single motion screw, determining the turning radius for path tracking. It is more advanced than RPP in the sense that it also takes the path heading into account.

Controller performance depends on the platform, computation, and scenario. DWB, RPP, and VP were tested to identify the best fit for the UGV.

4. Experimental Results

4.1. ROS2 Control Validation

All features rely on the assumption that velocity commands are correctly executed. Figure 5 compares the velocity commands with the odometry estimates from SLAM trial 18.

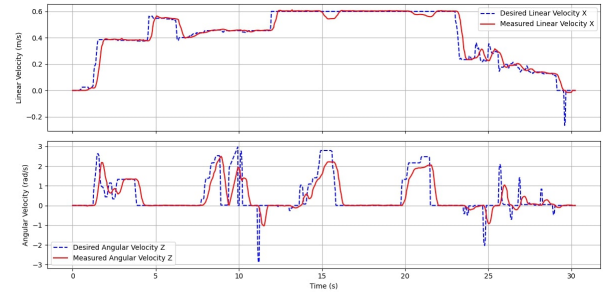


Figure 5: Velocity commands vs measured velocity.

The motor response to incoming commands is generally fast, exhibiting minimal delay under typical conditions. However, when command inputs change rapidly, limitations due to motor acceler-

ation become apparent. Around seconds 16 and 21, linear velocity drops coincide with unmet peaks in angular velocity. This occurred since the right wheel received a peak command of 41.397 rad/s, beyond the motor's physical limit.

4.2. Obstacle Avoidance Algorithm Testing

Two scenarios were designed to test the collision avoidance algorithm (figure 6).

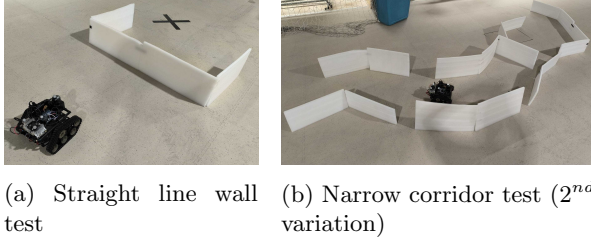


Figure 6: Collision avoidance test scenarios.

In both, the UGV was teleoperated forward at 0.6m/s with zero angular velocity, simulating the use case where the user focuses on covering ground efficiently. In the first scenario, the UGV was driven straight towards a wall 2 m ahead. The second scenario tested teleoperation through a narrow corridor. In a variation, a wall blocked the corridor entirely, validating the algorithm's ability to stop the robot when Zone 1 is triggered. Multiple trials were conducted without collisions.

Figure 7 shows the UGV's trajectory in a trial of the wall test. Even though zones are defined by LiDAR distance, the figure illustrates how in Zone 3 corrections are minor and delayed due to high initial velocity, while Zone 2 corrections are stronger, effectively redirecting the UGV.

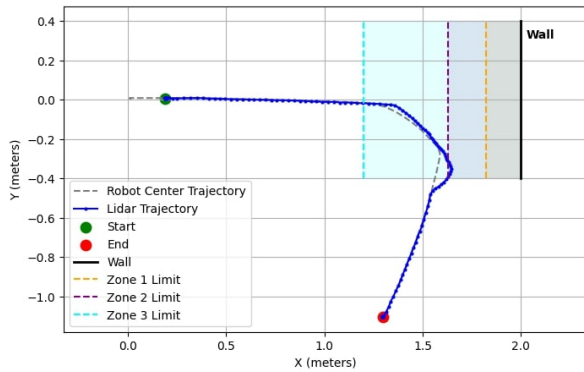


Figure 7: UGV trajectory during the wall test.

Fig. 8 shows user and corrective velocity commands during a corridor test trial (2nd variation). It illustrates how corrective commands activate as soon as the robot enters a restricted zone. In Zone 1 corrective commands override the user input to enforce a full stop.

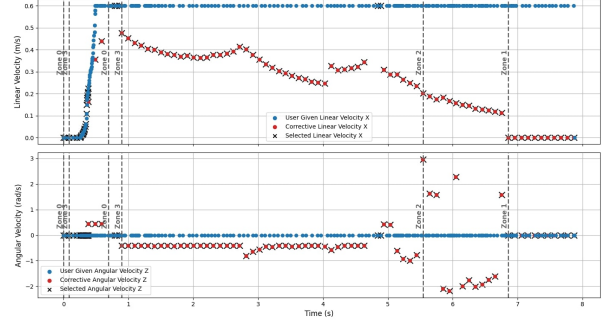


Figure 8: Velocity commands during the blocked corridor test.

4.3. SLAM Evaluation

4.3.1 SLAM Map Evaluation

Outdoor testing was performed to test the SLAM implementation mapping capabilities, using the scenario in Figure 9, chosen for its defined boundaries and challenging terrain. A mobile hotspot enabled remote operation despite added network latency.

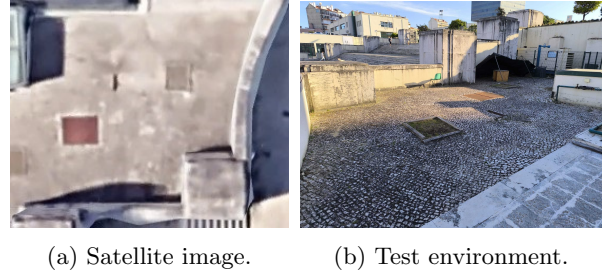


Figure 9: Outdoor test scenario.

Satellite imagery (Fig. 10a) was processed (edge detection, blur, dilation, binarization) to create a ground truth map (Fig. 10b).

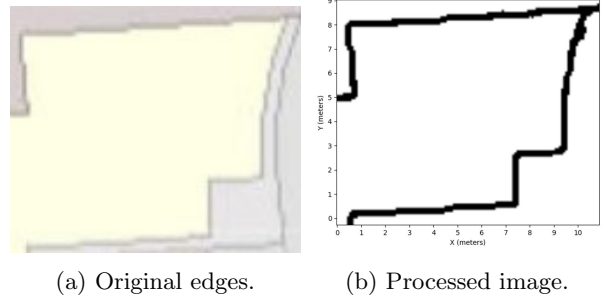


Figure 10: Ground truth map creation.

Map alignment was performed using Open3D's RANSAC global registration, fine tuning parameters via grid search. The alignment process minimized the average distance to the nearest neighbor, computed as $ADNN = \frac{1}{N} \sum_{i=1}^N d_i$, where d_i is the distance from a ground truth point to the near-

est SLAM map point. Fig. 11 shows the map obtained from SLAM Toolbox and its alignment with the ground truth.

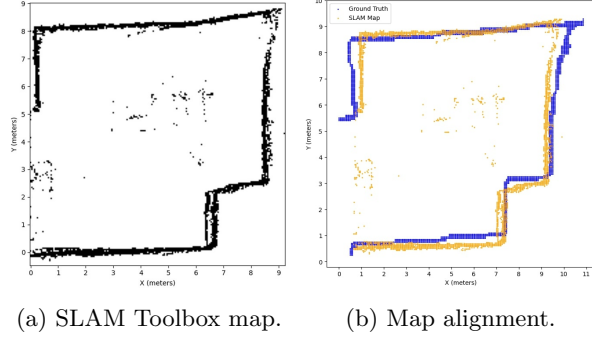


Figure 11: SLAM map and alignment.

A ADNN of 0.164 m was achieved. Visual misalignments suggest orientation errors, likely caused by the scenario’s lack of distinctive features and the specialization of LiDAR for indoor use.

4.3.2 SLAM Localization Evaluation

Localization accuracy was tested in the Robotics Arena at Instituto Superior Técnico, equipped with a Qualisys 3D infrared tracking system, serving as ground truth. An obstacle course was assembled (Fig. 12), and the UGV was teleoperated in it while both the Qualisys system and SLAM Toolbox recorded its pose.

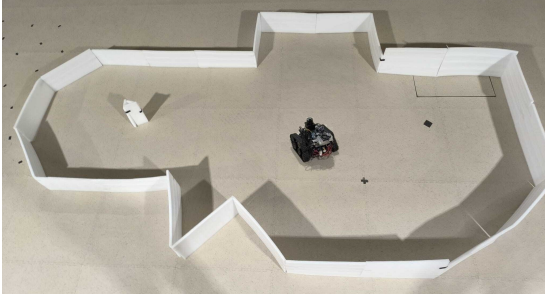


Figure 12: Obstacle course in Robotics Arena.

To ensure reference frames alignment, the robot was set to always start at the origin of the Qualisys system. In the first trajectory, the robot completed a single lap. This setup was tested over 15 trials. Fig. 13 shows a trial’s resulting map and trajectory.

A second trajectory (10 trials) involved two laps to evaluate if extended operation improved mapping and reduced pose errors. The position error was computed using the Euclidean distance, while the orientation error was calculated using wrapped yaw difference: $e_\theta = ((\theta_{\text{slam}} - \theta_{\text{gt}}) + 180^\circ) \bmod 360^\circ - 180^\circ$. Overall trial error was measured using $\text{RMSE}(e) = \sqrt{\frac{1}{N} \sum_{i=1}^N e_i^2}$

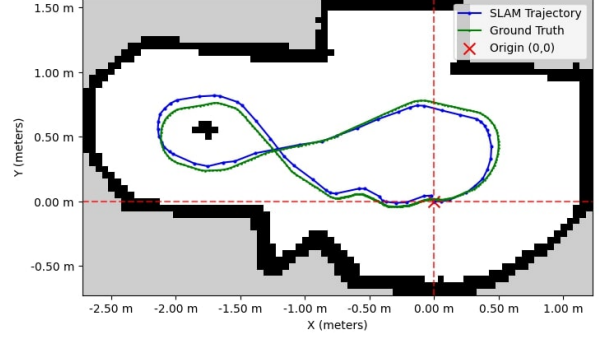


Figure 13: Trajectory comparison: Qualisys vs SLAM Toolbox.

The best map over the 25 trials was selected. In a third set of 10 trials, this map was reused in localization mode. In each trial, the robot performed three laps to test localization consistency throughout longer operation times. Figure 14 shows the RMSE across the three trial sets. Key findings:

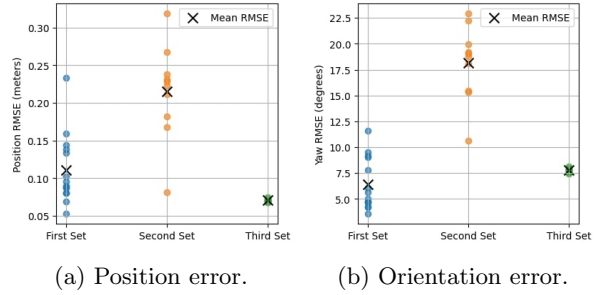


Figure 14: RMSE across the three trial sets.

More laps degraded mapping, as seen in the second set which yielded worse RMSE values. Sharp turns caused odometry drift and scan misalignment, resulting in tilted maps and loop closure failure, with the exception of trial 18 which successfully performed loop closure. Localization mode showcased consistently low RMSE values.

4.3.3 SLAM Computational Cost

SLAM Toolbox’s computational demand was a limiting factor. To maintain real-time performance, parameters were adjusted, such as lowering map resolution and the frequency of scans processed. To quantify resource usage, CPU and memory consumption were monitored during all 35 trials using Python’s `psutil`, sampled once per second and averaged per trial. CPU usage is expressed as a percentage of a single core; memory as the percentage of total RAM (7.7 GB) on the Jetson TX2.

Table 2 shows memory usage remained low across all trials. CPU usage increased slightly in the second set, reflecting the longer paths and denser pose-

graphs typical of graph-based SLAM. In the third set, a notable CPU rise occurred due to the switch to localization mode, where frequent scan matching against a full prebuilt map, and more laps, increased computational demands.

Table 2: Mean ($\bar{x}$) and standard deviation (σ) of CPU and memory usage by trial group. All values are in percentage.

Trials	$\bar{x}_{\text{CPU}}$	σ_{CPU}	$\bar{x}_{\text{Mem}}$	σ_{Mem}
1 st Set	24.10	6.73	0.51	0.02
2 nd Set	31.18	3.85	0.52	0.03
3 rd Set	79.71	6.81	0.62	0.02

4.4. Waypoint Navigation Validation

To evaluate the waypoint navigation solution, experiments were conducted in the Robotics Arena, with obstacles forming narrow passages (Fig. 15).



Figure 15: Obstacle course assembled.

The scenario was first mapped using manually placed waypoints in Rviz, all of which were successfully reached, confirming functionality in unknown areas. The resulting map was saved and loaded with SLAM Toolbox and Nav2 to form the global costmap. Two sets of 10 waypoints were randomly generated using a Python script with constraints: bounding the region (6*2.5 m), enforcing navigability using a cost threshold, and maintaining a minimum distance between consecutive waypoints of 1 m, to ensure meaningful paths. Each set was executed with three different configurations, consisting of the NavFn planner with the DWB, RPP and VP controllers. All 60 waypoints were reached with no collisions, yielding a 100% success rate.

The Theta* planner was also tested with the DWB controller. Although it avoided collisions, its tendency to route close to obstacles often triggered recovery behaviours and significantly increased execution time. As a result, NavFn was preferred.

Fig. 16 illustrates the initial state of the global costmap with the waypoints and paths planned during the first trial.

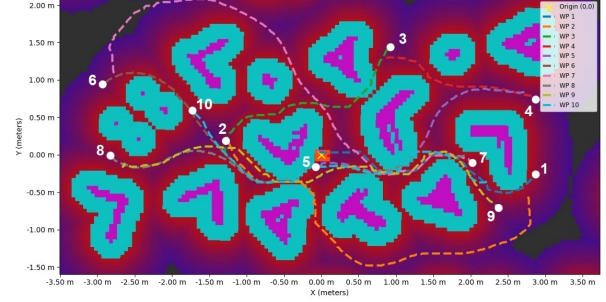


Figure 16: Global costmap of the obstacle course with waypoints and initial paths.

The costmap and paths dynamically update during execution. Thus, the actual reference path differs slightly from earlier ones. Additionally, due to goal tolerance and the robot's motion characteristics, there is often a small offset between a waypoint and the start of the next path. This reduces experimental repeatability. Path consistency was evaluated through the planned path lengths shown in Fig. 17. Most paths remained consistent across trials, though some variations were observed.

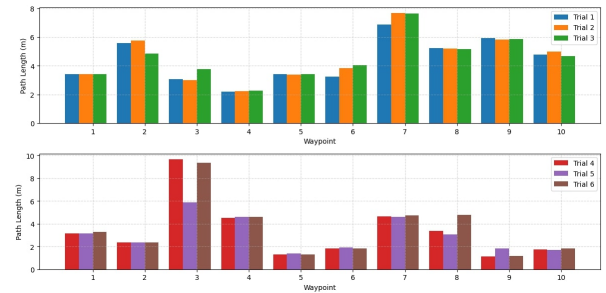


Figure 17: Planned path length across different waypoints and trials.

4.4.1 Path Planner Evaluation

Planning time was measured from the moment a goal pose was received to the moment an executable path was generated. The results across all trials are shown in Fig. 18.

A recurring issue was the immediate abortion of goals, causing replanning and inflating average planning times. Overall, the average was 0.425 s, but the 0.008 s median shows path generation was nearly instantaneous under normal conditions.

4.4.2 Local Planners Evaluation

To evaluate the local planners' path-tracking performance, the position of the UGV was compared

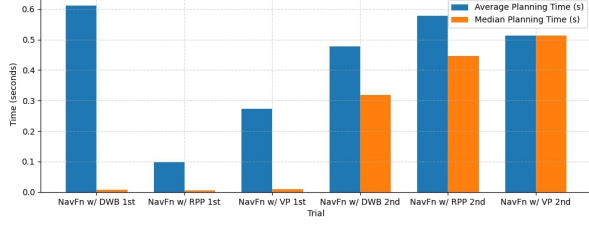


Figure 18: Planning time per trial.

with the reference path. As paths are dynamically updated, deviation was measured relative to the active path at the moment each pose was recorded. Each path was resampled via linear interpolation to ensure uniform spatial resolution. For each recorded pose (x_i, y_i) , the lateral error (e) was defined as the shortest Euclidean distance to the interpolated path P' , computed using a k-dimensional tree for nearest-neighbor search:

$$e = \min_{(x_p, y_p) \in P'} \|(x_i, y_i) - (x_p, y_p)\|_2 \quad (9)$$

Figure 19 illustrates this metric using data from trial 3.

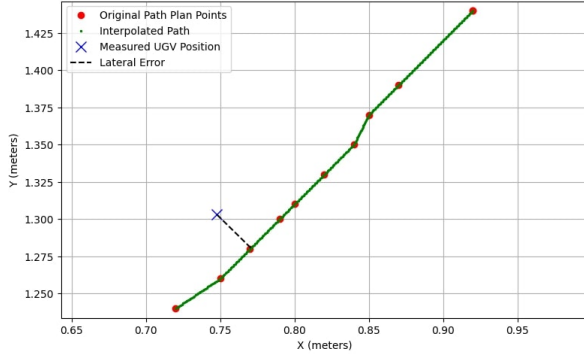


Figure 19: Lateral error calculation from robot pose to nearest point on interpolated path.

Table 3 shows the mean, standard deviation, and median of the lateral error for each trial, along with total distance travelled.

The DWB controller consistently showed slightly lower lateral errors than the geometric controllers (RPP and VP), which is unexpected. Given their design focus on minimizing path deviation, one would anticipate better performance from the geometric controllers. Nevertheless, the global average lateral error of 0.037 m across all trials indicates accurate and reliable path tracking.

Total distance travelled is another relevant metric. While a shorter path might suggest efficiency, it could also imply riskier trajectories closer to obstacles. In both waypoint sets, the DWB controller travelled the shorter distance, consistent with its

Table 3: Lateral error metrics and distance travelled per trial. All values are in meters.

Trial	$\bar{x}_e$	σ_e	Med $_e$	Travel Dist.
DWB 1 st	0.034	0.030	0.027	45.110
RPP 1 st	0.039	0.032	0.033	45.587
VP 1 st	0.038	0.034	0.029	46.376
DWB 2 nd	0.033	0.030	0.027	32.035
RPP 2 nd	0.034	0.030	0.026	32.850
VP 2 nd	0.039	0.042	0.029	44.719
Global	0.037	0.034	0.029	—

tendency to cut corners. The VP controller had the longest total distance, especially in the second set, where recovery behaviours inflated this metric.

Execution time is another key performance indicator. Fig. 20 breaks down each trial into planning, execution, and idle time. Execution time spanned from path generation to waypoint arrival. Idle time refers to the delay between reaching a waypoint and initiating planning for the next, deliberately added to ensure the UGV came to a full stop, improving the observability of waypoint transitions.

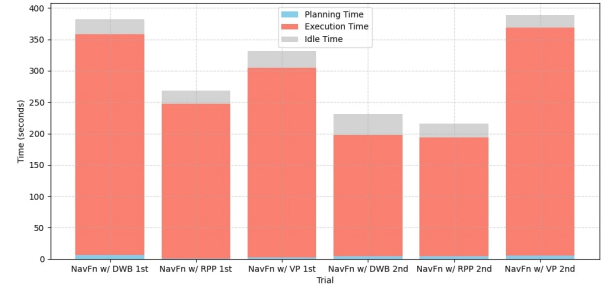


Figure 20: Breakdown of planning, execution, and idle time for each trial.

The RPP controller achieved the lowest overall trial time across both waypoint sets. In contrast, the DWB and VP controllers exhibited at least one trial with significantly longer durations, primarily due to increased recovery behaviours.

4.4.3 Navigation Computational Cost

As with SLAM Toolbox, Nav2 required compromises to manage computational load, including reducing server update rates and lowering costmap resolution. Table 4 shows the CPU and memory usage, monitored using Python's `psutil` library.

Memory usage remained negligible. In contrast, CPU usage was high, averaging approximately 145%, with peaks reaching just above 200%.

Table 4: Mean ($\bar{x}$) and standard deviation (σ) of CPU and memory usage across all trials. All values are in percentage.

Trial	$\bar{x}_{\text{CPU}}$	σ_{CPU}	$\bar{x}_{\text{Mem}}$	σ_{Mem}
All Trials	145.38	12.00	3.78	0.11

5. Conclusions

All project objectives outlined in Section 1 were successfully accomplished. A modular and robust UGV was developed, featuring a compact hardware architecture and a containerized ROS2-based software stack supporting teleoperation, autonomous waypoint navigation, and collision avoidance. The platform’s modular design and software containerization ensure ease of maintenance, reproducibility, and scalability for future deployments and research.

A low-level control interface enabled precise motor actuation from velocity commands, structured through a priority-based command multiplexer. Teleoperation was implemented using a DualShock 4 gamepad. A reactive collision avoidance module was developed to enhance operational safety while maintaining user control.

SLAM capabilities, implemented using SLAM Toolbox, were validated qualitatively and quantitatively in indoor and outdoor settings, showing low localization RMSE and accurate mapping via the ADNN metric. Waypoint navigation, enabled through the Nav2 framework, achieved 100% success across 60 randomly assigned goals using three local planners, without any collisions.

References

- [1] Q. Liu, Z. Li, S. Yuan, Y. Zhu, and X. Li. Review on vehicle detection technology for unmanned ground vehicles. *Sensors*, 2021.
- [2] A. Machado, A. Moutinho, M. Pinho, and T. Silva. Unmanned ground vehicles development in nato. In *International Scientific Conference” Strategies XXI”*, 2024.
- [3] European Commission. Annex to the commission implementing decision on the financing of the european defence fund and adoption of the work programme for 2024 - part ii, with amendments to previous work programmes (2021, 2022, 2023). EDF 2024 URL. Pages 126-137. Accessed: 2024-08-21.
- [4] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall. Robot operating system 2: Design, architecture, and uses in the wild. *Science Robotics*, 2022.
- [5] Open Source Robotics Foundation. Solidworks urdf exporter. https://github.com/ros/solidworks_urdf_exporter. Accessed: 2024-09-15.
- [6] J. Newans. ros_arduino_bridge - ros package for arduino communication. https://github.com/joshnewans/ros_arduino_bridge. Accessed: 2024-09-15.
- [7] R. Siegwart, Illah. R. Nourbakhsh, and D. Scaramuzza. *Introduction to Autonomous Mobile Robots*. The MIT Press, 2nd edition, 2011. ISBN 0262015358.
- [8] S. Chitta, E. Marder-Eppstein, W. Meeussen, V. Pradeep, A. Rodríguez Tsouroukdissian, J. Bohren, D. Coleman, B. Magyar, G. Raiola, M. Lüdtke, and E. Fernández Perdomo. ros_control: A generic and simple control framework for ros. *The Journal of Open Source Software*, 2017.
- [9] Buzzology. diffdrive_arduino - differential drive robot controller with arduino. https://github.com/Buzzology/diffdrive_arduino/tree/main. Accessed: 2024-09-15.
- [10] ROS 2 Navigation Stack Contributors. Navigation2: Collision monitor. https://github.com/ros-navigation/navigation2/tree/main/nav2_collision_monitor, 2025. Accessed: February 15, 2025.
- [11] V. Kumar. Turtlebot3 obstacle avoidance ros2. <https://github.com/vinay06vinay/Turtlebot3-Obstacle-Avoidance-ROS2/tree/main>. Accessed: February 15, 2025.
- [12] Open Source Robotics Foundation. twist_mux package overview - ros 2 documentation. https://index.ros.org/p/twist_mux/#humble-overview. Accessed: 2025-04-28.
- [13] S. Macenski and I. Jambrecic. Slam toolbox: Slam for the dynamic world. *Journal of Open Source Software*, 2021.
- [14] S. Macenski, F. Martín, R. White, and J. Ginés Clavero. The marathon 2: A navigation system. In *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020.
- [15] S. Macenski, T. Moore, D. V. Lu, A. Merzlyakov, and M. Ferguson. From the desks of ros maintainers: A survey of modern capable mobile robotics algorithms in the robot operating system 2. *Robotics and Autonomous Systems*, 2023.